

Question1:

Part A:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int i;

    char *commands[] = { "ls", "date", "whoami" };

    for (i = 0; i < 3; i++) {
        pid = fork();
        if (pid < 0) {
            perror("fork failed");
            exit(1);
        } else if (pid == 0) {
            // Child process executes a command
            execlp(commands[i], commands[i], NULL);
            perror("execlp failed"); // exec only returns on failure
            exit(1);
        }
    }

    // Parent waits for all children
    for (i = 0; i < 3; i++) {
        int status;
        pid_t child_pid = wait(&status);
        if (WIFEXITED(status)) {
            printf("Child %d exited with status %d\n", child_pid,
WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("Child %d was killed by signal %d\n", child_pid,
WTERMSIG(status));
        }
    }
}
```

```

    }
}

return 0;
}

```

A terminal window titled 'lab@rawal-lab1-pc-01: ~' showing the following commands and output:

```

lab@rawal-lab1-pc-01:~$ gcc partA.c -o t1
lab@rawal-lab1-pc-01:~$ ./t1
15:15:28 و PKT 2026 مئی 05 ت
lab
Downloads Lab partA.c partB.c partC.c Screenshots snap t1 task1.c
Child 5226 exited with status 0
Child 5227 exited with status 0
Child 5225 exited with status 0
lab@rawal-lab1-pc-01:~$

```

Part B:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(1);
    }

    pid_t pid;
    char *commands[] = { "ls", "date", "whoami" };

    for (int i = 0; i < 3; i++) {
        pid = fork();
        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }
    }
}

```

```

    } else if (pid == 0) {
        // Child closes read end
        close(pipefd[0]);
        // Redirect stdout to write end of pipe
        dup2(pipefd[1], STDOUT_FILENO);
        dup2(pipefd[1], STDERR_FILENO); // optional: redirect stderr too
        close(pipefd[1]);
        execlp(commands[i], commands[i], NULL);
        perror("execlp failed");
        exit(1);
    }
}

// Parent closes write end
close(pipefd[1]);

char buffer[1024];
ssize_t nbytes;
// Read all output from children
while ((nbytes = read(pipefd[0], buffer, sizeof(buffer)-1)) > 0) {
    buffer[nbytes] = '\0';
    printf("%s", buffer);
}
close(pipefd[0]);

// Wait for children
for (int i = 0; i < 3; i++) {
    int status;
    wait(&status);
}

return 0;
}

```

```
lab@rawal-lab1-pc-01: ~  
lab@rawal-lab1-pc-01:~$ gcc partB.c -o t2  
lab@rawal-lab1-pc-01:~$ ./t2  
15:17:53 و PKT 2026 مئی 05 ت  
lab  
Downloads  
Lab  
partA.c  
partB.c  
partC.c  
Screenshots  
snap  
t1  
t2  
task1.c  
lab@rawal-lab1-pc-01:~$
```

Part C:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <signal.h>
```

```
void sigchld_handler(int sig) {  
    int status;  
    pid_t pid;  
    // Use waitpid with WNOHANG to avoid blocking  
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {  
        if (WIFEXITED(status)) {  
            printf("SIGCHLD: Child %d exited with status %d\n", pid,  
WEXITSTATUS(status));  
        } else if (WIFSIGNALED(status)) {  
            printf("SIGCHLD: Child %d killed by signal %d\n", pid,  
WTERMSIG(status));  
        }  
    }  
}
```

```
int main() {  
    signal(SIGCHLD, sigchld_handler); // register handler
```

```

int pipefd[2];
if (pipe(pipefd) == -1) {
    perror("pipe failed");
    exit(1);
}

char *commands[] = { "ls", "date", "whoami" };

for (int i = 0; i < 3; i++) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp(commands[i], commands[i], NULL);
        perror("execlp failed");
        exit(1);
    }
}

close(pipefd[1]); // parent closes write end
char buffer[1024];
ssize_t nbytes;
while ((nbytes = read(pipefd[0], buffer, sizeof(buffer)-1)) > 0) {
    buffer[nbytes] = '\0';
    printf("%s", buffer);
}
close(pipefd[0]);

// Keep parent alive until all children terminate
while (wait(NULL) > 0);

return 0;

```

```

}
segmentation fault (core dumped)
lab@rawal-lab1-pc-01:~$ gcc partC.c -o t3
lab@rawal-lab1-pc-01:~$ ./t3
15:19:16 و PKT 2026 ت 05 مئی
lab
SIGCHLD: Child 5483 exited with status 0
SIGCHLD: Child 5484 exited with status 0
Downloads
Lab
partA.c
partB.c
partC.c
Screenshots
snap
t1
t2
t3
task1.c
lab@rawal-lab1-pc-01:~$

```

Question 2:

sender.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

#define FIFO_PATH "./chat_fifo"
#define MSG_COUNT 5
#define DELAY_SEC 1

int main() {
    // Create the named pipe (FIFO)
    if (mkfifo(FIFO_PATH, 0666) == -1) {
        if (errno != EEXIST) {
            perror("mkfifo failed");
            exit(EXIT_FAILURE);
        }
        // FIFO already exists – continue
    }
}

```

```

    fprintf(stderr, "Sender: using existing FIFO '%s'\n", FIFO_PATH);
} else {
    fprintf(stderr, "Sender: created FIFO '%s'\n", FIFO_PATH);
}

// Open FIFO for writing (blocks until a reader connects)
int fd = open(FIFO_PATH, O_WRONLY);
if (fd == -1) {
    perror("open FIFO for writing");
    exit(EXIT_FAILURE);
}
fprintf(stderr, "Sender: receiver connected. Sending %d messages...\n", MSG_COUNT);

// Send messages
for (int i = 1; i <= MSG_COUNT; i++) {
    char buffer[256];
    snprintf(buffer, sizeof(buffer), "Message %d from sender: Hello, this is chat message
#%d\n", i, i);

    ssize_t written = write(fd, buffer, strlen(buffer));
    if (written == -1) {
        perror("write to FIFO");
        close(fd);
        exit(EXIT_FAILURE);
    }

    printf("Sender: sent -> %s", buffer); // visible on sender's terminal
    sleep(DELAY_SEC);
}

close(fd);
fprintf(stderr, "Sender: all messages sent. Closing.\n");
// NOTE: do NOT unlink the FIFO here; the receiver will clean it up.
return 0;
}

```

```
lab@rawal-lab1-pc-01:~$ ./sender
Sender: created FIFO './chat_fifo'
Sender: receiver connected. Sending 5 messages...
Sender: sent -> Message 1 from sender: Hello, this is chat message #1
Sender: sent -> Message 2 from sender: Hello, this is chat message #2
Sender: sent -> Message 3 from sender: Hello, this is chat message #3
Sender: sent -> Message 4 from sender: Hello, this is chat message #4
Sender: sent -> Message 5 from sender: Hello, this is chat message #5
Sender: all messages sent. Closing.
lab@rawal-lab1-pc-01:~$
```

receiver.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

#define FIFO_PATH "./chat_fifo"
#define LOG_FILE "chat_log.txt"

// Global variables for cleanup (needed in signal handlers)
static int fifo_fd = -1;
static FILE *fifo_stream = NULL;
static const char *fifo_path = FIFO_PATH;

// Cleanup function: close FIFO, remove the named pipe, and exit
static void cleanup_and_exit(int exit_code) {
    if (fifo_stream) {
        fclose(fifo_stream);
        fifo_stream = NULL;
    }
    if (fifo_fd != -1) {
        close(fifo_fd);
        fifo_fd = -1;
    }
    unlink(fifo_path);
    exit(exit_code);
}
```

```

// Signal handler for SIGINT (Ctrl+C) and SIGPIPE
void signal_handler(int sig) {
    const char *msg;
    if (sig == SIGINT)
        msg = "\nReceiver: SIGINT received. Goodbye!\n";
    else if (sig == SIGPIPE)
        msg = "\nReceiver: SIGPIPE received (sender died). Goodbye!\n";
    else
        msg = "\nReceiver: signal received. Exiting.\n";

    // Write goodbye message to stderr (still visible after stdout redirection)
    write(STDERR_FILENO, msg, strlen(msg));

    // Clean up and exit
    cleanup_and_exit(EXIT_SUCCESS);
}

int main() {
    // 1. Open the named pipe for reading (created by sender)
    fifo_fd = open(FIFO_PATH, O_RDONLY);
    if (fifo_fd == -1) {
        perror("open FIFO for reading");
        fprintf(stderr, "Make sure the sender is running first (it creates the FIFO).\n");
        exit(EXIT_FAILURE);
    }
    fprintf(stderr, "Receiver: connected to FIFO '%s'\n", FIFO_PATH);

    // 2. Redirect stdout to the log file using dup2()
    int log_fd = open(LOG_FILE, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (log_fd == -1) {
        perror("open log file");
        close(fifo_fd);
        exit(EXIT_FAILURE);
    }
    if (dup2(log_fd, STDOUT_FILENO) == -1) {
        perror("dup2 failed");
        close(log_fd);
        close(fifo_fd);
        exit(EXIT_FAILURE);
    }
    close(log_fd); // original fd no longer needed
    fprintf(stderr, "Receiver: stdout redirected to '%s'\n", LOG_FILE);

    // 3. Convert FIFO file descriptor to a FILE stream for easy line reading

```

```

fifo_stream = fdopen(fifo_fd, "r");
if (!fifo_stream) {
    perror("fdopen");
    close(fifo_fd);
    exit(EXIT_FAILURE);
}

// 4. Install signal handlers
struct sigaction sa;
sa.sa_handler = signal_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
if (sigaction(SIGINT, &sa, NULL) == -1) {
    perror("sigaction SIGINT");
}
if (sigaction(SIGPIPE, &sa, NULL) == -1) {
    perror("sigaction SIGPIPE");
}
fprintf(stderr, "Receiver: signal handlers installed (SIGINT, SIGPIPE).\n");

// 5. Read messages from the FIFO and write them to stdout (now the log file)
char buffer[1024];
fprintf(stderr, "Receiver: waiting for messages...\n");
while (fgets(buffer, sizeof(buffer), fifo_stream) != NULL) {
    // printf goes to chat_log.txt because stdout is redirected
    printf("%s", buffer);
    fflush(stdout); // ensure data is written immediately
}

// When the sender closes its end, fgets returns NULL -> EOF
fprintf(stderr, "Receiver: sender finished or FIFO closed. Exiting normally.\n");

// Cleanup and remove the named pipe
cleanup_and_exit(EXIT_SUCCESS);
return 0; // never reached
}

```

```
lab@rawal-lab1-pc-01:~$ ./receiver
Receiver: connected to FIFO './chat_fifo'
Receiver: stdout redirected to 'chat_log.txt'
Receiver: signal handlers installed (SIGINT, SIGPIPE).
Receiver: waiting for messages...
Receiver: sender finished or FIFO closed. Exiting normally.
lab@rawal-lab1-pc-01:~$
```

Question 3:

// Part (a): No synchronization (Race Condition Demonstration)

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int balance = 5000;

void* deposit(void* arg) {
    for(int i = 0; i < 1000; i++) {
        balance += 100;    // NOT SAFE
    }
    return NULL;
}

void* withdraw(void* arg) {
    for(int i = 0; i < 1000; i++) {
        balance -= 100;    // NOT SAFE
    }
    return NULL;
}

int main() {
    pthread_t t[6];

    // 3 deposit + 3 withdraw threads
    for(int i = 0; i < 3; i++) {
        pthread_create(&t[i], NULL, deposit, NULL);
        pthread_create(&t[i+3], NULL, withdraw, NULL);
    }

    for(int i = 0; i < 6; i++) {
        pthread_join(t[i], NULL);
    }
}
```

```

    printf("Final Balance (No Sync): %d\n", balance);
    return 0;
}

// Part (b): Fix using Mutex

#include <stdio.h>
#include <pthread.h>

int balance = 5000;
pthread_mutex_t lock;

void* deposit(void* arg) {
    for(int i = 0; i < 1000; i++) {
        pthread_mutex_lock(&lock);
        balance += 100;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

void* withdraw(void* arg) {
    for(int i = 0; i < 1000; i++) {
        pthread_mutex_lock(&lock);
        balance -= 100;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t[6];
    pthread_mutex_init(&lock, NULL);

    for(int i = 0; i < 3; i++) {
        pthread_create(&t[i], NULL, deposit, NULL);
        pthread_create(&t[i+3], NULL, withdraw, NULL);
    }

    for(int i = 0; i < 6; i++) {
        pthread_join(t[i], NULL);
    }

    printf("Final Balance (With Mutex): %d\n", balance);

    pthread_mutex_destroy(&lock);
    return 0;
}

```

```
}
```

// Part (c): Add Semaphore (3 teller windows)

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

int balance = 5000;
pthread_mutex_t lock;
sem_t teller;

void* deposit(void* arg) {
    sem_wait(&teller); // Enter bank

    for(int i = 0; i < 1000; i++) {
        pthread_mutex_lock(&lock);
        balance += 100;
        pthread_mutex_unlock(&lock);
    }

    sem_post(&teller); // Leave bank
    return NULL;
}

void* withdraw(void* arg) {
    sem_wait(&teller);

    for(int i = 0; i < 1000; i++) {
        pthread_mutex_lock(&lock);
        balance -= 100;
        pthread_mutex_unlock(&lock);
    }

    sem_post(&teller);
    return NULL;
}

int main() {
    pthread_t t[6];
    pthread_mutex_init(&lock, NULL);
    sem_init(&teller, 0, 3); // Only 3 threads allowed

    for(int i = 0; i < 3; i++) {
        pthread_create(&t[i], NULL, deposit, NULL);
        pthread_create(&t[i+3], NULL, withdraw, NULL);
    }
}
```

```

    for(int i = 0; i < 6; i++) {
        pthread_join(t[i], NULL);
    }

    printf("Final Balance (With Semaphore): %d\n", balance);

    pthread_mutex_destroy(&lock);
    sem_destroy(&teller);
    return 0;
}

```

 shah@DESKTOP-02Q2MPC: ~

```

shah@DESKTOP-02Q2MPC:~$ gcc q3_part1.c -o part1 -pthread
shah@DESKTOP-02Q2MPC:~$ ./part1
Final Balance (No Sync): 5000
shah@DESKTOP-02Q2MPC:~$ gcc q3_part2.c -o part2 -pthread
shah@DESKTOP-02Q2MPC:~$ ./part2
Final Balance (With Mutex): 5000
shah@DESKTOP-02Q2MPC:~$ gcc q3_part3.c -o part3 -pthread
shah@DESKTOP-02Q2MPC:~$ ./part3
Final Balance (With Semaphore): 5000
shah@DESKTOP-02Q2MPC:~$ gcc q3_part4.c -o part4 -pthread
shah@DESKTOP-02Q2MPC:~$ ./part4
Audit Thread: Balance after all deposits = 105000
shah@DESKTOP-02Q2MPC:~$

```